

Today's agenda

↳ Quick revision

↳ is BitSet

↳ unique element 1

↳ unique element 2

Just for today's class → Instructors

Intro?

Subhesh

PTU

Subhesh → google

5+ on codechef {java}

ICPC → regional

3 yos+ teaching comp.

youtube content creator

level 3

CP

→ Graph

level 2

> 1m views

→ Reproding

Instructor & Sde → Scales

* Decimal to binary

↳ $(12)_{10} \rightarrow$

2	12
2	6
2	3
2	1
	0

0
0
1
1

$$= (1100)_2$$

↓

$$2^0 \cdot 0 + 2^1 \cdot 0 +$$

$$2^2 \cdot 1 + 2^3 \cdot 1 = 12$$

↳ $(45)_{10} \rightarrow$

2	45
2	22
2	11
2	5
2	2
2	1
	0

1
0
1
1
0
1

$$= (101101)_2$$

$$(1011010)_2$$

→

$$2^1 + 2^3 + 2^4 + 2^6 = 90$$

Bitwise operations

A	B	0 dominant	1 dominant	Same Same PAPA Shame
		$A \& B$	$A B$	$A \wedge B$
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Ex:

$$\begin{array}{rcl}
 A = 20 & : & \begin{array}{cccccc} \text{5} & \text{4} & \text{3} & \text{2} & \text{1} & \text{0} \\ 0 & 1 & 0 & 1 & 0 & 0 \end{array} \\
 B = 45 & : & \begin{array}{cccccc} 1 & 0 & 1 & 1 & 0 & 1 \\ \hline 0 & 0 & 0 & 1 & 0 & 0 \end{array} = 4
 \end{array}$$

$$\begin{array}{rcl}
 A = 20 & : & \begin{array}{cccccc} \text{5} & \text{4} & \text{3} & \text{2} & \text{1} & \text{0} \\ 0 & 1 & 0 & 1 & 0 & 0 \end{array} \\
 B = 45 & : & \begin{array}{cccccc} 1 & 0 & 1 & 1 & 0 & 1 \\ \hline 1 & 1 & 1 & 0 & 0 & 1 \end{array} = 57
 \end{array}$$

→ Print (20 & 45); → 4

Q) For given N check if i^{th} bit is Set or Not?

Ex: $N = 53$:
 6 5 4 3 2 1 0 $i = 3$
 0 1 1 0 1 0 1

↳ false

$N = 53$:
 6 5 4 3 2 1 0 $i = 2$
 0 1 1 0 1 0 1

↳ true

$N = 53$:
 6 5 4 3 2 1 0 $i = 3$
 0 1 1 0 1 0 1

$(1 \ll i)$:
 0 0 0 1 0 0 0 $= 0$

0 0 0 0 0 0 0 $\Rightarrow$ bit is unset.

$N = 53$:
 6 5 4 3 2 1 0 $i = 2$
 0 1 1 0 1 0 1

$(1 \ll i)$:
 0 0 0 0 1 0 0 $= 0$

0 0 0 0 1 0 0 $\Rightarrow$ bit is set.

false $\rightarrow$ unset
 true $\rightarrow$ set

T.C: $O(1)$

S.C: $O(1)$

```
boolean checkBit (int N, int i) {
```

```
    if (N & (1 << i) == 0) {
```

```
        return false;
```

```
    }
```

```
    else
```

```
        return true;
```

```
}
```

}

Q) Every element repeats twice except 1, find unique element.

Ex: arr[]: {3 2 3 7 2 8 7} $\rightarrow$ ans=8

$$A \oplus A = 0$$

$$A \oplus 0 = A$$

$$A \oplus B = B \oplus A$$

arr[]: {3 2 3 7 2 8 7}

$\hookrightarrow$ Take xor of all elements.

$$3 \oplus 2 \oplus 3 \oplus 7 \oplus 2 \oplus 8 \oplus 7$$

$$3 \oplus 3 \oplus 2 \oplus 2 \oplus 7 \oplus 7 \oplus 8$$

$$0 \oplus 0 \oplus 0 \oplus 8 = 8$$

```
int ans = arr[0];
```

```
for (int i = 1; i < N; i++) {
```

```
    ans = ans ^ arr[i];
```

```
}
```

```
return ans;
```

T.C: $O(N)$

S.C: $O(1)$

$$\rightarrow 2 \oplus 3 \oplus 2 \oplus 3 \Rightarrow 0$$

$$\rightarrow 2 \oplus 2 \oplus 3 \oplus 3 \Rightarrow 0$$

Q) unique element II

↳ Given $arr[N]$, every element repeats thrice except

1, find the unique element?
↳ Comes 1 time

$arr[7]:$

0	1	2	3	4	5	6
6	5	6	4	5	6	5

 $\rightarrow ans: 4$

$arr[13]:$

0	1	2	3	4	5	6	7	8	9	10	11	12
5	7	5	4	7	11	11	9	11	7	5	4	4

 $\rightarrow ans: 9$

// Brute force

↳ use hashmap (Count occ. of each number)

T.C: $O(N)$

S.C: $O(N)$

$O(1)$

// Taking XOR

$$A \wedge A \wedge A = A$$

↳
 $arr[13]:$

0	1	2	3	4	5	6	7	8	9	10	11	12
5	7	5	4	7	11	11	9	11	7	5	4	4

$$\rightarrow 9 \wedge 11 \wedge 7 \wedge 5 \wedge 4 = ??$$

Hint $\rightarrow$ Counting bits of bit indexes

// optimal idea

arr[13] = 5 7 5 4 7 11 11 9 11 7 5 4 4

$\leftarrow i$
 5 → 0 0 0 1 0 1
 7 → 0 0 0 1 1 1
 5 → 0 0 0 1 0 1
 4 → 0 0 0 1 0 0
 7 → 0 0 0 1 1 1
 11 → 0 0 1 0 1 1
 11 → 0 0 1 0 1 1
 9 → 0 0 1 0 0 1
 11 → 0 0 1 0 1 1
 7 → 0 0 0 1 1 1
 5 → 0 0 0 1 0 1
 4 → 0 0 0 1 0 0
 j → 4 → 0 0 0 1 0 0
 5 → 0 0 0 1 0 0

ans
 4 9 6 10
 $4 \% 3 = 1$ $9 \% 3 = 0$ $6 \% 3 = 0$ $10 \% 3 = 1$
 ans: 0 1 0 0 1

```

int uniqueelement2 (int arr[N]) {
    int ans = 0;
    for (int i = 0; i < 32; i++) {
        int count = 0;
        for (int j = 0; j < N; j++) {
            if (checkBit(arr[j], i) == true)
                count++;
        }
        if (count % 3 == 1) {
            ans = ans + (1 << i);
        }
    }
    return ans;
}
  
```

ans = $0 + 2^0 + 2^3 = 9$

i = 0 → Count: 4 $\% 3 = 1$

T.C: $O(32 * N) \approx O(N)$

S.C: $O(1)$

//Pseudo code

```
int uniqueelement2 (int arr[N]) {
```

```
    int ans = 0;
```

```
    for (int i = 0; i < 32; i++) {
```

```
        int count = 0;
```

```
        for (int j = 0; j < N; j++) {
```

```
            if (checkBit(arr[j], i) == true) {
```

```
                count ++;
```

```
            }
```

```
        if (count % 3 == 1) {
```

```
            ans = ans + (1 << i);
```

```
        return ans;
```

```
    }
```

```
boolean checkBit (int N, int i) {
```

```
    if (N & (1 << i) == 0) {
```

```
        return false;
```

```
    } else
```

```
        return true;
```

```
}
```


Extension:

↳ Every element is repeating twice, except 1 element.

↳ repeating 2 times

3 3 3 4 4 4 2 2

$3 * N$

↓
0 → bit is unset

$3 * N + 2$

↓
1 → bit is set

11 Count % 3 == ~~2~~ 2

↳ Every element is repeating 4 times except 1.

↳ Count % 4 == 1 ⇒ T.C: $O(32 * N) = O(N)$

↳ XOR of all elements. ⇒ T.C: $O(N)$

↳ Every element is repeating 4 times except 1

↳ occ. 2 times

↳ XOR won't work.

↳ Count % 4 == 2

Break till 10:34 pm

Q) unique element III

↳ Given $arr[n]$, every element repeats twice except 2 elements. find the 2 unique elements.

↳ each occ.
1 time

Ex: $arr[6] = \{ \overset{0}{3} \overset{1}{6} \overset{2}{4} \overset{3}{4} \overset{4}{3} \overset{5}{8} \}$: ans: {6, 8}

$arr[4] = \{ \overset{0}{4} \overset{1}{9} \overset{2}{9} \overset{3}{8} \}$: ans: {4, 8}

//idea1

↳ use Hashmap.

T.C: $O(N)$

S.C: $O(N)$

//idea2

↳ xor of all numbers

$\{ \overset{0}{\cancel{3}} \overset{1}{6} \overset{2}{\cancel{4}} \overset{3}{\cancel{4}} \overset{4}{\cancel{3}} \overset{5}{8} \}$

= 14 → { can we get 2 unique elements? }

arr[12] = 10 8 8 9 12 9 6 11 10 6 12 17

¹⁰¹⁰ ¹⁰⁰⁰ ¹¹⁰⁰ ⁰¹¹⁰ ¹⁰¹⁰ ¹¹⁰⁰
₁₀₀₀ ₁₀₀₁ ₁₀₀₁ ₁₀₁₁ ₀₁₁₀ ₁₀₀₀₁

n1: 0 1 1

n2: 1 0 0

26: 1 1 0 1 0

4 3 2 1 0

On the basis of 3rd bit index
divide the array

unset

set

unset = $0 \wedge 6 \wedge 6 \wedge 17 = 17$

set = $0 \wedge 10 \wedge 8$

$18 \wedge 9 \wedge 12 \wedge 9 \wedge 11$

$\wedge 10 \wedge 12 = 11$

Claim!: one set bit will be there.

(i)

(ii)

// Pseudo code

```
int uniqueelement3 (int arr[N]) {
```

Step 1:

```
    int v = arr[0];  
    for (int i = 1; i < N; i++) {  
        v = v ^ arr[i];  
    }
```

T.C: $O(N) + O(32)$
 $\downarrow$
 $O(N)$

S.C: $O(1)$

Step 2:

```
    int pos = 0;  
    for (int i = 0; i < 32; i++) {  
        if (checkBit(v, i) == true) {  
            pos = i;  
            break;  
        }  
    }
```

Step 3:

```
    int ans1 = 0, ans2 = 0;
```

```
    for (int i = 0; i < N; i++) {  
        if (checkBit(arr[i], pos) == true) {  
            ans1 = ans1 ^ arr[i];  
        }  
        else {  
            ans2 = ans2 ^ arr[i];  
        }  
    }  
    print(ans1);
```

Point (ans 2):

↳ 11th & 12th → DTU

↳ remember/memorize

↓
1 ↓ 2 ↓ 3 ↓
IIT delhi BITS Pilani Nit

understanding

45 mins

→ (1) understand → memorize → revision

↳ Prefix Sum

repeating stuff & find unique

↳ PSP ≥ 95%

↳ 2000+

1000 leetcode
+ 300

codechef + codeforces